Student notes — Session 3 · Data types, comparisons and if

Slides: [`Week_02_Programming_Principles_I_C.pdf`](../slides/Week_02_Programming_Principles_I_C.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Week 2: Data types and conditionals

What this week is about. Last week your programs always did the same thing: read a number, calculate, print. This week your programs start to **decide**. Look at the code on the slide: if the temperature is above 38, print "Check fever". If not, print nothing. By Friday you will write programs that do different things for different inputs.

Quiz 1 is on Friday 18 September, in the first 25 minutes of class. It is on paper, no laptop. It covers **week 1 only**: variables and their types, `printf` and its holes, `scanf` and the `&`, `+ - * / %`, the math functions, and reading or writing a small program like the circle calculator. One or two short questions. The best five of the six quizzes count, so there is no make-up quiz. The best way to prepare is Knowledge checks 1 and 2 and the practice tests on Blackboard.

How to try the examples in these notes. Same as last week:

1. Open Visual Studio Code. Click **File**, then **Open Folder**, and open your `c_week1` folder (or make a new one called `c_week2`).
2. Click **File**, then **New Text File**. Type the program. Click **File**, then **Save As**, and give it a name ending in `.c`, for example `test.c`.
3. Click **View**, then **Terminal**. In the terminal type `gcc -Wall test.c -o test` and press Enter. If nothing is printed, it worked.
4. Type `./test` and press Enter. (In Windows PowerShell, type `.\test` instead.)

Many examples below are only a few lines. To run them, put the lines inside this frame, where the `...` is:

```
#include <stdio.h>
int main(void) {
    ...
    return 0;
}
```

Slide 2 — Today's roadmap

Six steps this week:

1. Why data types exist.
2. The number types: `int` and `double` (and `float`, which we do not use).
3. `char`: one character.
4. What happens when types mix, and the trap in it.
5. Yes/no in C: comparisons and `&&`, `||`, `!`.
6. Decisions: `if`, `else`, `else if`, `switch`.

Tuesday's class covers steps 1 to 5 and the start of step 6 (`if` and `if / else`). Friday, after the quiz, does the rest.

The program at the bottom of the slide is where Tuesday ends:

```
int age = 18;
double price = 12.75;
```

```
char grade = 'A';
if (age >= 18) {
    printf("Adult ticket");
} else {
    printf("Student ticket");
}
```

It prints `Adult ticket`, because 18 is "greater or equal to" 18. Change `age` to 17 and it prints `Student ticket`. By the end of these notes you will understand every line.

Slide 3 — Why do we need data types?

Inside the computer, everything is a row of zeros and ones. The row on the slide, `0 1 0 0 0 0 0 1`, could be the number 65. It could be the letter A. The same bits. **The type tells C what the bits mean.**

The type also tells C how big the box must be. A whole number needs some space. A number with a decimal point needs more. When you write the type, C reserves the right amount.

Four kinds of value from a student record:

Value	Kind	Example	Type
Student ID	whole number	20261234	`int`
GPA	decimal number	3.74	`double`
Letter grade	one symbol	A	`char`
Passed?	yes or no	1 or 0	`int` (see slide 9)

The question on the slide. What type for a student's age, the tuition amount, and the final letter grade? Age: `int`. Tuition: `double`, because it can be 1250.50. Letter grade: `char`.

Habit. Before you write a variable, ask: "what kind of thing is this?" A count, a measurement, or a symbol? Then the type follows.

Slide 4 — Core C data types

This table is the slide. You will use three of the four rows.

Type	Use it for	Example value	printf hole	scanf hole
`int`	whole numbers: counts, ages, credits	`35`	`%d`	`%d`
`double`	numbers with a decimal point: money, GPA, measurements	`4.50`	`%f` or `%.2f`	`%lf`
`char`	one character: a grade, a menu letter	`'B'`	`%c`	`%c`
`float`	decimal number, smaller box, fewer digits	`3.74`	`%f`	`%f`

The rule: counts are `int`, decimals are `double`, one symbol is `char`.

About `float`. You will see `float` in books and on the internet. It also holds a decimal number, but in a smaller box, so it keeps fewer digits. In this course, **do not use `float`**. Use `double`. Here is why:

```
float f = 1.0 / 3;
double d = 1.0 / 3;
printf("%.10f\n", f);
printf("%.10f\n", d);
```

The screen shows:

```
0.3333333433
0.3333333333
```

The `float` goes wrong after seven digits. The `double` is right.

Example: one variable of each type.

```
int students = 35;
double price = 4.50;
char section = 'B';
printf("%d %.2f %c\n", students, price, section);
```

The screen shows `35 4.50 B`.

Careful, again: reading a `double` with `scanf` is `%lf`. Printing it is `%f`. This is the mistake most people made on Friday.

Try it yourself. Which type would you use for each of these? (Answers below.)

1. The number of people at a table.
2. The bill in manat.
3. The letter of your section.
4. The temperature outside.
5. How many days until the exam.

(Answers: `int`, `double`, `char`, `double` because it can be 21.5, `int`.)

Slide 5 — Declaring and initializing variables

Two words:

- **Declaring** makes the box. `int count;`
- **Initialising** makes the box and puts the first value in, on the same line. `int students = 35;`

Both are allowed. But there is a trap between them.

Example 1. Declared and initialised. All three of these are safe to use straight away:

```
int students = 35;
double price = 4.50;
char section = 'B';
```

Example 2. Declared first, assigned later. Also fine, as long as you assign before you use:

```
int count;
count = 10;
printf("%d\n", count);
```

The screen shows `10`.

Example 3. Declared and never assigned. This is the trap:

```
int x;
printf("%d\n", x);
```

What is inside `x`? Not zero. **Garbage.** Whatever was lying in that part of the memory before. It might print `0`. It might print `4198256`. It might print something different tomorrow. When you compile this with `-Wall`, the compiler warns you:

```
warning: 'x' is used uninitialized
```

When you see that warning, you read a box before you put anything in it.

Rule: give every box a value before you read it. The easiest way is to initialise when you declare: `int count = 0;`.

The question on the slide. Which variables are safe to print immediately? `students`, `price` and `section`, because they were initialised. `count` only after the line `count = 10;`. `x` never, because nothing was ever put in it.

Try it yourself.

1. Type Example 3, compile it with `-Wall`, and read the warning.
2. Change `int x;` to `int x = 0;`. Compile again. The warning is gone and the program prints `0`.

Slide 6 — Integer division: the first serious trap

You saw this on Friday: `7 / 3` is 2, not 2.33. Here is the rule and what it is good for.

The rule. When both numbers are `int`, the answer is an `int`. The part after the decimal point is **thrown away**. Not rounded. Thrown away.

```
int total = 7;
int people = 2;
printf("%d\n", total / people);
printf("%d\n", total % people);
```

The screen shows:

```
3
1
```

`7 / 2` is 3.5, and the .5 is thrown away, so `3`. `7 % 2` is the remainder: 2 goes into 7 three times, and 1 is left, so `1`.

Think of it as one division with two answers: 7 = 3 × 2 + 1. The `/` gives you the 3. The `%` gives you the 1.

Expression	Result	Why
`7 / 2`	`3`	3.5, and the .5 is thrown away
`7 % 2`	`1`	3 × 2 = 6, and 1 is left
`130 / 60`	`2`	60 goes into 130 two times
`130 % 60`	`10`	2 × 60 = 120, and 10 is left
`9 % 2`	`1`	odd numbers leave 1
`8 % 2`	`0`	even numbers leave 0

The example on the slide: hours and minutes.

```
int minutes = 130;
printf("%d h %d min\n", minutes / 60, minutes % 60);
```

The screen shows `2 h 10 min`. The `/` gives the hours, the `%` gives the minutes left over. That is what `%` is for: hours and minutes, minutes and seconds, is a number even or odd, what is the last digit of a number.

Why it is a trap. The compiler says nothing. `7 / 2` is not an error and not a warning. It is 3, quietly. You have to know the rule. The cure is on the next slide.

Try it yourself. Write down what you think each line prints. Then run it.

1. `printf("%d\n", 200 / 60);`
2. `printf("%d\n", 200 % 60);`
3. `printf("%d min %d s\n", 125 / 60, 125 % 60);`

(Answers: `3`, `20`, `2 min 5 s`.)

Slide 7 — Type conversion

This is the cure for integer division, and the mistake almost everyone makes on the way.

The key sentence: C calculates the expression first, then stores the result into the variable.

```
int a = 7;
int b = 2;
double result1 = a / b;
double result2 = (double)a / b;
printf("%.2f\n", result1);
printf("%.2f\n", result2);
```

The screen shows:

```
3.00
3.50
```

Why `result1` is 3.00. C looks at `a / b` first. Both are `int`, so it is integer division: 3. *Then* it puts 3 into the `double` box `result1`. A 3 in a `double` box shows as `3.00`. The box being a `double` came too late. The half was already thrown away.

Why `result2` is 3.50. `(double)a` is a **cast**. A type in round brackets in front of a value means "treat this value as that type, just here". So `(double)a` is 7.0. Then `7.0 / 2`: one side is a `double`, so C keeps the decimals: 3.5. Then 3.5 is stored.

The second mistake: casting the answer instead of a number.

```
double result3 = (double)(a / b);
printf("%.2f\n", result3);
```

The screen shows `3.00`. The brackets say "do `a / b` first". That is integer division, 3. The cast turns 3 into 3.0. Too late again. **The cast must touch one of the two numbers before the division, not the answer after it.**

Line	Result	Why
`double r = a / b;`	`3.00`	division first, as `int`, then stored
`double r = (double)a / b;`	`3.50`	`a` becomes 7.0, then decimal division
`double r = a / (double)b;`	`3.50`	`b` becomes 2.0, same thing
`double r = 1.0 * a / b;`	`3.50`	`1.0 * a` is 7.0, then decimal division
`double r = (double)(a / b);`	`3.00`	division first, cast after: too late

The other direction. Putting a decimal into an `int` box throws the decimals away, and `-Wall` does not warn:

```
int whole = 3.99;
printf("%d\n", whole);
```

The screen shows `3`. Putting a whole number into a `double` box is harmless: `double d = 5;` holds `5.00`.

Try it yourself. With `int x = 9;` and `int y = 4;`, write down what each prints, then run it.

1. `printf("%d\n", x / y);`
2. `printf("%.2f\n", (double)x / y);`
3. `printf("%.2f\n", (double)(x / y));`

(Answers: `2`, `2.25`, `2.00`.)

Slide 8 — Characters: char

A `char` holds **one** character. One letter, one digit, one symbol. It is written in **single quotes**:

```
char grade = 'A';
char answer = 'y';
char digit = '7';
printf("%c\n", grade);
printf("%c %c\n", answer, digit);
```

The screen shows:

```
A
y 7
```

Print a `char` with `%c`.

Single quotes, not double. `'A'` is a `char`. `"A"` with double quotes is a **string**, a piece of text, which comes next week. If you write `char grade = "A";`, the compiler warns `makes integer from pointer without a cast` and the program prints rubbish. That warning means: wrong quotes.

`'7'` is not `7`. These two look the same on the screen but are different things:

```
char digit = '7';
int number = 7;
```

`number` is a quantity. You can add to it: `number + 1` is 8. `digit` is a symbol, a picture of a seven. **The question on the slide:** which one can be used directly in arithmetic? `number`.

What is really inside a `char`. Every symbol is stored as a number. Print a `char` with `%d` and you see that number:

```
printf("%d\n", '7');
printf("%d\n", 'A');
```

The screen shows `55` and `65`. 55 is the number for the symbol 7, and 65 for the symbol A. That is the row of bits on slide 3. Next week we use this. For now, remember only: **`'7'` is a symbol, `7` is a number, and single quotes make a `char`.**

Try it yourself.

1. Make `char c = 'Q';` and print it with `%c`. (Expect `Q`.)
2. Print the same `c` with `%d`. (Expect `81`.)

3. Make `char d = '5';` and `int n = 5;` and print `n + 1` with `%d`. (Expect `6`.)

Slide 9 — Comparisons produce yes/no answers

A program decides by asking questions. Is the age 18 or more? Is the score under 50? Each question has a yes/no answer.

In C, yes is 1 and no is 0. A comparison is a small calculation whose answer is 1 or 0. You can print it with `%d`.

The six ways to ask:

Symbol	Question	Example	Answer
`>`	greater than?	`19 > 18`	`1`
`>=`	greater or equal?	`18 >= 18`	`1`
`<`	less than?	`10 < 4`	`0`
`<=`	less or equal?	`7 <= 7`	`1`
`==`	equal?	`19 == 18`	`0`
`!=`	not equal?	`5 != 3`	`1`

The example on the slide.

```
int age = 19;
printf("%d\n", age >= 18);
printf("%d\n", age == 18);
```

The screen shows:

```
1
0
```

Is 19 greater or equal to 18? Yes: `1`. Is 19 equal to 18? No: `0`.

The question on the slide. What is `5 != 3`? Is 5 different from 3? Yes: `1`.

The bug of the semester: `=` and `==`.

- `x = 5` **puts** 5 into `x`. One equals sign. It is a command.
- `x == 5` **asks** "is `x` equal to 5?". Two equals signs. It is a question.

If you write `=` where you meant `==`, the program still compiles and does something, just not what you wanted. With `-Wall` you get a hint:

```
warning: suggest parentheses around assignment used as truth value
```

When you see that warning, look for a `=` that should be `==`.

Try it yourself. Write down what each prints, then run it.

1. `printf("%d\n", 3 > 2);`
2. `printf("%d\n", 7 != 7);`
3. `printf("%d\n", 40 > 38);`
4. `int n = 9; printf("%d\n", n % 2 == 1);`

(Answers: `1`, `0`, `1`, `1`. The last one asks "is the remainder of 9 ÷ 2 equal to 1?", and it is, so 9 is odd.)

Slide 10 — Combining conditions

Real questions have more than one part. "Can enter the exam" is: registered **and** not late **and** has an ID card. C has three joiners.

- `&&` is **AND**: both sides must be yes. `3 > 2 && 2 > 1` is `1`. `3 > 2 && 2 > 5` is `0`, because the second half is no.
- `||` is **OR**: at least one side must be yes. `3 > 2 || 2 > 5` is `1`, because the first half is yes.
- `!` is **NOT**: it flips the answer. `!0` is `1`, and `!1` is `0`.

The `|` is the vertical bar, usually on the key above Enter, with Shift. Type it twice.

The example on the slide.

```
int age = 20;
int hasCard = 1;
if (age >= 18 && hasCard) {
    printf("Allowed\n");
}
```

The screen shows `Allowed`. Is 20 at least 18? Yes, 1. Is `hasCard` yes? It is 1. Both yes, so the whole condition is yes.

Notice `hasCard` stands alone, with no comparison. It is already a 1 or a 0, so it is already an answer. That is how you store a yes/no: an `int` with 1 for yes and 0 for no.

Change `hasCard` to 0. Now the condition is "yes and no", which is no, and nothing prints.

The question on the slide: a discount for a student or anyone under 18.

```
if (isStudent || age < 18) {
    printf("Discount\n");
}
```

`isStudent`	`age`	The condition	Prints
1	30	yes or no = yes	`Discount`
0	16	no or yes = yes	`Discount`
0	30	no or no = no	nothing

`!` (not). It flips a yes/no. It reads best in front of a yes/no box:

```
int raining = 0;
if (!raining) {
    printf("Go outside\n");
}
```

The screen shows `Go outside`. `raining` is 0, no. `!raining` is 1, yes.

Try it yourself. Write each rule in C.

1. Can drive: age is 18 or more **and** has a licence (`hasLicence` is 1 or 0).
2. Free entry: under 6 **or** 65 and over.
3. Not logged in (`loggedIn` is 1 or 0).

(Answers: `age >= 18 && hasLicence`, `age < 6 || age >= 65`, `!loggedIn`.)

Slide 11 — The if statement

An `if` statement runs a block of lines only when a question's answer is yes.

The program on the slide.

```
#include <stdio.h>
int main(void) {
    int score;
    printf("Score: ");
    scanf("%d", &score);
    if (score >= 50) {
        printf("Passed\n");
    }
    printf("Done\n");
    return 0;
}
```

Line by line.

- `if (score >= 50) {` reads as "if the score is at least 50, then". The question goes in round brackets. Then an open curly brace.
- `printf("Passed\n");` is the **block**: what to do when the answer is yes.
- `}` closes the block.
- `printf("Done\n");` is **after** the closing brace, so it is not part of the `if`. It runs every time.

The two questions on the slide.

Run it and type `49`. The screen shows:

```
Score: Done
```

Is 49 at least 50? No, so the block is skipped, and only `Done` prints.

Run it and type `50`. The screen shows:

```
Score: Passed
Done
```

Is 50 at least 50? Yes, so `Passed` prints, and then `Done` prints as always.

The picture on the slide. The condition is a fork in the road. Yes: go through the block. No: skip around it. Then both roads join again and the program carries on.

Shape rules.

- No semicolon after the `)` of the `if`. The braces are its body.
- Indent the block four spaces. The compiler does not care. You will, when there are three `if`s inside each other.
- Even if the block is one line, **use the braces**. Without them, only the very next line belongs to the `if`, and it is easy to get wrong.

Try it yourself.

1. Change `50` to `60`. Run it with 55. (Expect `Done` only.)
2. Add a second `if` after the first: if the score is 90 or more, print `Excellent`. Run it with 95. (Expect `Passed`, `Excellent`, `Done`.)

Slide 12 — if / else

`if` on its own does something or nothing. Usually you want **one of two things**. That is `else`.

The program on the slide.

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Adult ticket\n");
    } else {
        printf("Student ticket\n");
    }
    return 0;
}
```

You type	Screen shows	Why
`20`	`Adult ticket`	20 is at least 18: yes
`17`	`Student ticket`	17 is not: no, so `else`
`18`	`Adult ticket`	18 is "greater or equal": yes

The rule. If the condition is yes, the `if` block runs. If it is no, the `else` block runs. **Never both. Never neither.** Exactly one of the two, every time.

`else` has no question of its own. It means "otherwise". It catches everything the `if` did not. Do not write `else (age < 18)`: `else` takes no brackets and no condition. That is a compile error.

The mini-task on the slide. Change the program: under 6 is a free ticket, otherwise a normal ticket.

```
if (age < 6) {
    printf("Free ticket\n");
} else {
    printf("Normal ticket\n");
}
```

Type `4`: `Free ticket`. Type `6`: `Normal ticket`, because 6 is not under 6.

Usual mistakes.

- `>` instead of `<`: `if (age > 6)` gives free tickets to adults.
- `=` instead of a comparison: `if (age = 6)` puts 6 into `age` and then treats 6 as "yes". Look for the `-Wall` warning from slide 9.
- A semicolon after the `)`: `if (age < 6);` ends the `if` there, and the block runs always. `-Wall` warns `this 'if' clause does not guard...`.

Try it yourself.

1. Read a number. Print `Even` if `n % 2 == 0`, otherwise `Odd`. Test with 8 and 9.
2. Read a temperature as a `double`. Print `Check fever` if it is above 38.0, otherwise `Normal`. Test with 38.5 and 36.6.

Before Friday.

1. **Quiz 1 is Friday**, first 25 minutes, on paper, no laptop, on week 1 only. Do Knowledge checks 1 and 2 until your best is 4 of 6 or more, and the practice tests. The quiz asks the same kind of thing on paper: what does this print, find the mistake, write a short program.

2. **Do Knowledge check 3** on Blackboard, on today's slides. Zero percent, three attempts, best one counts.
3. **Try the new practice tests**: types and conversion, comparisons and logic, if and else.
4. **Bring the weekend programs** from last week, working or not.

Friday, after the quiz: `else if` chains, `switch`, and two whole programs, BMI and an electricity bill.